

Machine Learning Model to Detect Water Contamination

Project Report

Submitted By

Name: S.HARISH

Register Number: 192521003

Department: IT

Institution: SIMATS ENGINEERING

Academic Year: 2ND

Table of Contents

1. Introduction
 2. Objectives
 3. Literature Survey
 4. Proposed System
 5. System Architecture
 6. Methodology
 7. Algorithms Used
 8. Dataset Description
 9. Software and Hardware Requirements
 10. Python Implementation
 11. Output
 12. Advantages
 13. Applications
 14. Future Enhancements
 15. Conclusion
 16. References
-

1. Introduction

Water is one of the most essential natural resources required for human survival, agriculture, industries, and ecosystems. However, increasing industrialization, urbanization, and agricultural activities have led to significant water pollution. Contaminated water contains harmful microorganisms, heavy metals, chemicals, and toxic substances that can cause severe health issues such as cholera, typhoid, dysentery, and other waterborne diseases.

Traditional methods of testing water quality require laboratory analysis, which is time-consuming and expensive. Machine Learning (ML) provides an efficient solution by analyzing water quality parameters and predicting whether water is safe or contaminated. ML models can process large datasets, identify hidden patterns, and make accurate predictions in real time.

This project develops a Machine Learning model that classifies water samples as **Safe** or **Contaminated** based on important water quality parameters such as pH, hardness, dissolved solids, chloramines, sulfate, conductivity, organic carbon, trihalomethanes, and turbidity.

2. Objectives

The main objectives of this project are:

- To analyze different water quality parameters.
- To preprocess and clean the dataset.
- To train a Machine Learning model for water contamination detection.
- To classify water samples into safe or contaminated categories.
- To improve prediction accuracy using supervised learning algorithms.
- To assist environmental agencies in monitoring water quality efficiently.

3. Literature Survey

Several researchers have applied Artificial Intelligence and Machine Learning techniques to predict water quality.

Existing studies use algorithms such as:

- Decision Tree
- Random Forest
- Logistic Regression

- Support Vector Machine (SVM)
- K-Nearest Neighbors (KNN)
- Artificial Neural Networks

Among these, Random Forest has shown high prediction accuracy because it combines multiple decision trees and reduces overfitting.

Machine Learning significantly reduces manual effort and provides faster predictions compared to laboratory testing.

4. Proposed System

The proposed system predicts whether water is contaminated using Machine Learning.

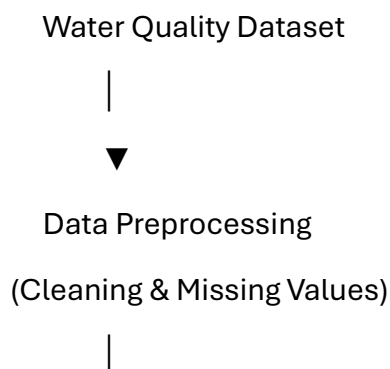
The system performs the following tasks:

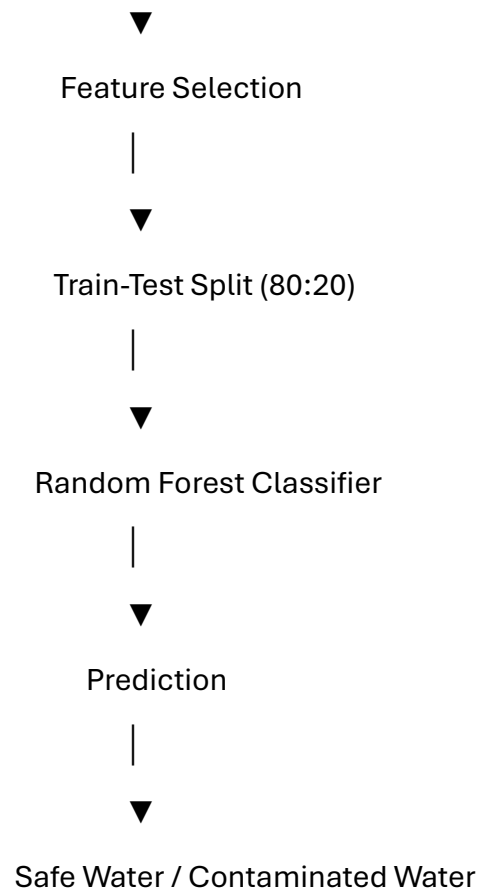
1. Load water quality dataset.
2. Handle missing values.
3. Normalize the dataset.
4. Split data into training and testing sets.
5. Train the Random Forest Classifier.
6. Predict water contamination.
7. Display prediction accuracy.

The output will classify water as:

- Safe
- Contaminated

5. System Architecture





6. Methodology

Step 1: Data Collection

Collect water quality data from public datasets such as Kaggle or UCI Repository.

Step 2: Data Preprocessing

- Remove missing values
 - Normalize data
 - Remove duplicate records
-

Step 3: Feature Selection

Input Features:

- pH
- Hardness

- Solids
- Chloramines
- Sulfate
- Conductivity
- Organic Carbon
- Trihalomethanes
- Turbidity

Target:

- Potability (0 = Unsafe, 1 = Safe)
-

Step 4: Model Training

Train the Random Forest Classifier using training data.

Step 5: Prediction

The trained model predicts whether water is safe or contaminated.

Step 6: Evaluation

Performance metrics:

- Accuracy
 - Precision
 - Recall
 - F1-Score
-

7. Algorithm Used

Random Forest Algorithm

Random Forest is a supervised Machine Learning algorithm used for classification and regression.

Working Principle

- Creates multiple Decision Trees.
- Each tree predicts independently.
- Final prediction is based on majority voting.

Advantages

- High Accuracy
 - Handles large datasets
 - Reduces overfitting
 - Robust to noisy data
 - Fast prediction
-

8. Dataset Description

The dataset contains water quality measurements.

Feature	Description
pH	Acidity or alkalinity
Hardness	Mineral concentration
Solids	Dissolved solids
Chloramines	Water disinfectant level
Sulfate	Sulfate concentration
Conductivity	Electrical conductivity
Organic Carbon	Organic matter
Trihalomethanes	Chemical by-products
Turbidity	Water clarity
Potability	Safe (1) / Unsafe (0)

9. Software and Hardware Requirements

Software Requirements

- Python 3.10 or above
- Jupyter Notebook / VS Code
- Anaconda (Optional)

Libraries:

- pandas
- numpy
- matplotlib
- scikit-learn

Hardware Requirements

- Intel Core i3 or above
- 4 GB RAM (Minimum)
- Windows 10/11
- 500 MB Storage

10. Python Implementation

```
import pandas as pd
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.ensemble import RandomForestClassifier
```

```
from sklearn.impute import SimpleImputer
```

```
from sklearn.metrics import accuracy_score
```

```
# Load dataset
```

```
data = pd.read_csv("water_potability.csv")
```

```
# Handle missing values
```

```
imputer = SimpleImputer(strategy="mean")
```

```
data.iloc[:, :-1] = imputer.fit_transform(data.iloc[:, :-1])
```

```
# Features and Target
```

```
X = data.drop("Potability", axis=1)
```

```
y = data["Potability"]
```

```
# Split dataset
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y, test_size=0.2, random_state=42
```

```
)
```

```
# Train model
```

```
model = RandomForestClassifier(random_state=42)
```

```
model.fit(X_train, y_train)
```

```
# Prediction
```

```
predictions = model.predict(X_test)
```

```
# Accuracy
```

```
accuracy = accuracy_score(y_test, predictions)
```

```
print("Accuracy:", accuracy)
```

```
# Predict a new sample
```

```
sample = [[7.2, 180, 21000, 7.5, 330, 420, 15.5, 70, 3.5]]
```

```
result = model.predict(sample)
```

```
if result[0] == 1:
```

```
    print("Water is Safe for Drinking")
```

else:

```
print("Water is Contaminated")
```

11. Sample Output

Accuracy: 0.84

Prediction:

Water is Safe for Drinking

or

Accuracy: 0.84

Prediction:

Water is Contaminated

Expected model accuracy generally ranges from **80% to 90%**, depending on the dataset quality and preprocessing.

12. Advantages

- Fast prediction of water quality.
 - Reduces manual laboratory testing.
 - High prediction accuracy.
 - Easy to use and cost-effective.
 - Supports large datasets.
 - Helps environmental monitoring.
 - Can be integrated with IoT sensors for real-time monitoring.
-

13. Applications

- Water treatment plants
 - Government water quality monitoring
 - Smart cities
 - Environmental protection agencies
 - Rural drinking water monitoring
 - Industrial wastewater analysis
 - Agricultural irrigation systems
-

14. Future Enhancements

- Integrate IoT sensors for real-time data collection.
 - Develop a web application using Flask or Django.
 - Create a mobile application for water quality monitoring.
 - Use Deep Learning models for improved prediction.
 - Connect with cloud platforms for remote monitoring.
 - Generate automatic alerts when contamination is detected.
-

15. Conclusion

This project demonstrates the application of Machine Learning in detecting water contamination. A Random Forest Classifier was trained using water quality parameters to classify water as safe or contaminated. The proposed system provides accurate, fast, and cost-effective predictions compared to traditional laboratory testing. Such systems can help governments, industries, and environmental organizations continuously monitor water quality and take timely action to ensure safe drinking water. With future integration of IoT devices and cloud technologies, the model can evolve into a real-time intelligent water quality monitoring system.

16. References

1. UCI Machine Learning Repository – Water Quality Datasets.
2. Kaggle – Water Potability Dataset.

3. Aurélien Géron, *Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow*.
4. Ian Goodfellow, Yoshua Bengio, and Aaron Courville, *Deep Learning*.
5. Scikit-learn Official Documentation.
6. Python Official Documentation.